

Group 7 Analytical Summary Report:
EdTech Student Learning Engagement & Course Completion Warehouse

I. Data Source

The project uses the anonymized Open University Learning Analytics Dataset (OULAD). It contains data about courses, students and their interactions with Virtual Learning Environment (VLE) for seven selected courses (called modules). Presentations of courses start in February and October – they are marked by “B” and “J” respectively. The dataset consists of tables connected using unique identifiers. All tables are stored in the csv format.

File courses.csv contains the list of all available modules and their presentations and has 22 rows. File assessments.csv contains information about assessments in module-presentations and has 206 rows. File vle.csv contains information about the available online materials in the VLE including html pages, pdf files, etc and has 6,364 rows. File studentInfo.csv contains demographic information about the students together with their results and has 32,593 rows. File studentRegistration.csv contains information about the time when the student registered for the module presentation and also has 32,593 rows. File studentAssessment.csv contains the results of students’ assessments and has 173,912 rows. Finally, studentVle.csv contains information about each student’s interactions with the materials in the VLE and has 10,655,282 rows.

The six relational files were stored in SQLite while the high-volume clickstream was stored in MongoDB. The grade records are tabular in nature and have fixed columns with clear relationships. The pipeline also required ACID transactions and joins and aggregations which meant the data is suited to be stored on a relational database. The clickstream logs on the other hand are event-based meaning that there is a high volume of data that needs to be read efficiently hence the use of MongoDB.

II. ETL Pipeline Design

landing_zone → extract (read_csv_file) → transform (transform_table) → quarantine → load (SQLite upsert / Mongo bucket insert) → verify → OLAP export → analysis

III. Verification Results

The SQLite verification results can be seen in the logs/milestone1_log.txt file with the MongoDB verification results are in data/MongoDB/mongodb_verification.json.

The verification process completed successfully with all checks passing. For the SQLite database, the row counts matched the loaded data for all six tables: dim_courses (22), dim_assessments (195), dim_vles (6,364), dim_students (32,593), fact_registration (32,593), and fact_assessment_results (171,047). All foreign key checks also passed, with 0 orphan rows found for every relationship, including

assessments-to-courses, VLEs-to-courses, students-to-courses, registrations-to-courses, registrations-to-students, and assessment results-to-assessments. The SQLite integrity_check returned OK, confirming that the database passed its integrity validation.

For the MongoDB database VLE_Logs, collection clickstream_events, the pipeline read 10,655,280 rows, of which 9,868,118 were valid and 8,459,320 remained after deduplication. A total of 1,960,496 documents were inserted, while 1,408,798 duplicate rows were skipped. The MongoDB collection contained exactly 1,960,496 documents, matching the number of inserted rows, so the document count check passed. The _id_ index was also confirmed to exist. Overall, all SQLite and MongoDB verification checks passed successfully, with no row-count mismatches, foreign key violations, database integrity errors, document count mismatches, or missing required indexes.

IV. OLAP Analysis Approach

a. Relational (SQLite)

The OLAP queries use window functions and analytical operations to examine student performance, assessment behavior, and engagement over time. The Student Progress Timeline query primarily uses the time-series/windowing OLAP operation through LAG(), cumulative SUM() OVER, and AVG() OVER to compare each assessment with the student's previous assessment and calculate cumulative scores and running averages. It answers questions such as: How has a student's score changed from one assessment to the next? How many assessments have they completed? What is their cumulative score and running average? How many days occur between submissions?

The Course Performance Summary uses ranking and aggregation as its core OLAP operations, combining RANK() OVER with grouped COUNT(), AVG(), SUM(), MAX(), and MIN() calculations. It ranks students within their course and presentation based on their average assessment score and calculates their relative percentile. This query answers: How does each student's performance compare with classmates? What is the student's average, total, highest, and lowest assessment score? What is their class rank and approximate percentile? How many students are in the course?

The Assessment Submission Patterns query uses ordering and partition-based window analysis through ROW_NUMBER(), AVG() OVER, and COUNT() OVER. Its core OLAP operation is submission sequencing and comparative aggregation, allowing submissions to be ordered by their submission date while also calculating the average score and total number of submissions for each assessment. It answers: Were students submitting assessments on time or late? How many days late were they? What is the average score for the assessment? In what order did submissions occur, and where does a particular submission fall within the overall submission sequence?

The Student Trajectory Analysis uses trend analysis and cumulative aggregation through FIRST_VALUE(), LAST_VALUE(), SUM() OVER, and COUNT()

OVER. Its core OLAP operation is longitudinal analysis, comparing a student's first and last assessment scores while calculating their running average throughout the course. It answers: What was the student's starting score? What was their final score? How much did their score improve or decline from the beginning? What is their running average over time? Is the student's overall trajectory improving, declining, or remaining stable?

The Time-Based Engagement query uses time-window analysis through DENSE_RANK() and a rolling window frame with AVG() OVER (... ROWS BETWEEN 3 PRECEDING AND CURRENT ROW). This allows student activity to be organized by submission week and evaluated using a rolling four-period average. It answers: How does a student's performance change across weeks? What is their recent rolling average? How many engagement events have accumulated over time? What are their minimum and maximum scores, and how does their current score compare with that performance range?

The Retention by Early Engagement query uses quartile segmentation and aggregation as its core OLAP operations. It first aggregates each student's early engagement using SUM() and then uses NTILE(4) OVER to divide students within each course and presentation into four engagement quartiles based on their early clicks. It then uses grouped aggregation to calculate the number of students, retention percentage, and pass percentage for each quartile. This query answers: How does early engagement differ across students? Do students with higher early engagement have higher retention rates? Do students with higher early engagement have higher Pass/Distinction rates? How does retention and academic success compare across the four early-engagement groups?

The Learning Path Lift query uses grouped aggregation and comparative OLAP analysis as its core operation. It aggregates early clicks by student, course, presentation, and activity type, identifies students who engaged with each activity, and then compares the Pass/Distinction rate of students who engaged with an activity against the overall baseline pass rate. The GROUP BY operation produces results for each activity type, while the calculated lift_vs_baseline measures how much the engaged group's pass rate differs from the overall baseline. This query answers: Which learning activities were students engaging with during the early period? What percentage of students who engaged with each activity achieved Pass or Distinction? How does the success rate of students who engaged with a particular activity compare with the overall student pass rate? Which activity types have the greatest positive or negative lift compared with the baseline?

b. Clickstream (MongoDB)

Three facets were generated from the reduced dataset due to the storage constraints therefore they do not represent the entire OULAD clickstream. These facets are totals, engagement_depth, and weekly_clicks. Facet totals is a summary of the total number of student-resource engagement pairs and the total clicks across all of them. It divides total clicks by total pairs to get the average number

of clicks per student-resource relationship, which can tell whether engagement across the platform tends to be visited once or twice or repeatedly. It shows how much clickstream activity exists overall, and how concentrated it is.

Facet engagement_dept buckets every student to resource relationship by how many distinct days it was active, then reports how many pairs and how many total clicks fall into each depth range. A distribution weighted heavily toward the lowest bucket (one or two active days) means most engagement across the platform is brief and non-repeating, while a meaningful population in the higher buckets shows the subset of student-resource relationships that reflect real, sustained use. Resources that repeatedly show up with deep engagement are the ones worth prioritizing, while resources where nearly all engagement is shallow may need to be redesigned or aren't serving a repeat-use function.

Facet weekly_clicks sums total clicks by week producing a timeline that can be plotted directly as a line chart. Spikes in the shape of that curve that align with assessment due dates confirm the data is behaving as expected, while a declining trend that isn't explained by assessment timing signals gradual disengagement across the student population. If click volume drops noticeably before withdrawal counts rise, that's evidence the clickstream can act as an early warning signal.